Fail-Closed Multi-Node Isolation & Routing Engineering Guide

Comprehensive Deployment Blueprint for VPN Gateway (VM1), Tor Middlebox (VM2), and Isolated Workstation (VM3)

1. Structural Network Topography

This implementation establishes an absolute fail-closed isolation pipeline designed to mitigate identity leaks during technical analysis. By utilizing discrete virtualization nodes, physical compartmentalization guarantees that even in the event of root compromise on the terminal execution environment, underlying cryptographic boundaries hold. No unencrypted or non-proxied packets can physically escape the architecture.

Machine Asset	Network Interface Context	Static IP Allocation	Primary Gateway Assignment
VM1: VPN Gateway	`ens18` (Proxmox `vmbr0`) `ens19` (Proxmox `vmbr1`) `wg0` (Virtual Tunnel)	DHCP (LAN Interface) `10.0.1.1/24` (Internal Core) Dynamic Assigned (VPN Node)	Upstream Physical Router IP None VPN Endpoint Interface Gateway
VM2: Tor Middlebox	`ens19` (Proxmox `vmbr1`) `ens20` (Proxmox `vmbr2`)	`10.0.1.2/24` (Upstream Internal) `10.0.2.1/24` (Downstream Segment)	`10.0.1.1` (VM1 Gateway Interface) None
VM3: Workstation	`ens18` (Proxmox `vmbr0`) `ens19` (Proxmox `vmbr2`)	Passive Maintenance Local Link `10.0.2.2/24` (Isolated Core)	None (Disabled for Net-Isolation) `10.0.2.1` (VM2 Proxy Interface)

> **CRITICAL HYPERVISOR BOUNDARY REQUIREMENTS**
>
> All virtual switches must strictly reflect this infrastructure layout. Proxmox interfaces attached to `vmbr1` and `vmbr2` must have the "Firewall" checkbox **unchecked** within the Proxmox Web GUI Hardware panel to bypass default hypervisor MAC filtering and stateless drop engines that cause asymmetric routing failure.

2. Machine 1 (VM1) — VPN Gateway Provisioning

VM1 functions as the hard network boundary, wrapping all upstream traffic from the lower layers inside an encrypted WireGuard tunnel. If the WireGuard link collapses, all downstream routing immediately terminates via UFW stateful drop configurations.

2.1 Core Network Forwarding Setup

The Linux kernel must explicitly enable IP forwarding to allow transit packets to pass across interfaces.

```
# Force immediate runtime activation of IPv4 transit forwarding
sudo sysctl -w net.ipv4.ip_forward=1
```

```
# Persist the structural variable across subsequent machine restarts
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

2.2 Persistent Netplan Infrastructure Configuration

Apply the dedicated static assignment on the private physical bridge `ens19` while keeping the primary management hook dynamic.

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Ensure the syntax strictly aligns as follows:

```
network:
  version: 2
  ethernets:
    ens18:
      dhcp4: true
    ens19:
      addresses:
        - 10.0.1.1/24
```

```
sudo netplan apply
```

2.3 WireGuard Profile Integration

Deploy your upstream configuration file directly to the cryptographic network endpoint path:

```
sudo nano /etc/wireguard/wg0.conf
```

Verify that your specific `[Interface]` block contains the explicit client declarations and `[Peer]` configuration endpoints. Once authenticated, trigger the interface up:

```
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

2.4 UFW Fail-Closed Firewall Rule Engine

We modify the default configuration parameters to isolate the private physical block `10.0.1.0/24` and bind its absolute exit authority entirely to the `wg0` interface.

```
sudo nano /etc/ufw/before.rules
```

Inject the mandatory Network Address Translation (NAT) engine block directly at the topmost position of the file (prior to any filter rule blocks):

```
*nat
:POSTROUTING ACCEPT [0:0]
-A POSTROUTING -s 10.0.1.0/24 -o wg0 -j MASQUERADE
COMMIT
```

Execute the core configuration changes and strict traffic forwarding commands:

```
# Reset global defaults to reject incoming and transit traffic
sudo ufw default deny incoming
sudo ufw default deny forward

# Inject precise route allowances for interface mapping
sudo ufw route allow in on ens19 out on wg0
sudo ufw allow in on ens19 to 10.0.1.1

# Enable firewall engine
sudo ufw enable
sudo ufw reload
```

3. Machine 2 (VM2) — Tor Middlebox Transparent Proxy Configuration

VM2 acts as the structural interceptor. It processes incoming data packets from VM3, strips their plain destination frames, and injects them seamlessly into localized Tor proxy sockets.

3.1 Netplan Infrastructure & WireGuard MTU Adaptation

To defeat the **WireGuard MTU Blackhole Phenomenon** (where encrypted overhead packets exceed standard 1500-byte frame lengths and fragment silently), the Maximum Transmission Unit on VM2's upstream interface must be constrained to exactly `1360` bytes.

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Populate the specific hardware interface matrix precisely:

```
network:
  version: 2
  ethernets:
    ens19:
      mtu: 1360
      addresses:
        - 10.0.1.2/24
      routes:
        - to: default
          via: 10.0.1.1
    ens20:
      addresses:
        - 10.0.2.1/24
```

```
sudo netplan apply
```

3.2 Secure Anti-Spoofing and Reverse Path Configuration

Because VM2 uses an internal route topology that returns traffic along asymmetric lines via the VPN proxy, the Linux kernel's default Reverse Path Filter (rp_filter) mechanism will interpret these packets as spoofed and drop them at layer 2. We must completely disable this protection across the interfaces.

```
sudo sysctl -w net.ipv4.conf.all.rp_filter=0
sudo sysctl -w net.ipv4.conf.default.rp_filter=0
sudo sysctl -w net.ipv4.conf.ens20.rp_filter=0

# Ensure persistence across manual reboots
sudo nano /etc/sysctl.d/99-tor-routing.conf
```

Append these exact variable assertions:

```
net.ipv4.conf.all.rp_filter=0
net.ipv4.conf.default.rp_filter=0
net.ipv4.conf.ens20.rp_filter=0
```

```
sudo sysctl --system
```

3.3 Tor Hardened Daemon Configuration

Configure the Tor daemon to serve as a network-wide DNS and Transparent Routing gateway, bound specifically to the static gateway interface.

```
sudo apt update && sudo apt install tor -y
sudo nano /etc/tor/torrc
```

Append the exact parameters below, stripping away any generic configurations:

```
VirtualAddrNetworkIPv4 10.192.0.0/10
AutomapHostsOnResolve 1
TransPort 10.0.2.1:9040
DNSPort 10.0.2.1:5353
```

Wipe any previously corrupted bootstrap state parameters and trigger a clean execution initialization:

```
sudo systemctl stop tor
sudo rm -rf /var/lib/tor/cached-*
sudo rm -f /var/lib/tor/state
sudo systemctl start tor
```

3.4 Pure Transparent Interception Netfilter Script (IPTables)

This script establishes a complete block against normal cleartext data routing. It traps all inbound DNS requests on UDP port 53 and web strings on TCP, pushing them cleanly into Tor's isolated listeners.

```
# Flush existing network rules completely
sudo iptables -F
sudo iptables -t nat -F
sudo iptables -X

# Establish Localhost (Loopback) operational baseline allowances
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

```
# Permit established tracking rules for operational traffic
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Permit Local Inbound management commands from VM3 interface explicitly
sudo iptables -A INPUT -i ens20 -j ACCEPT

# Micro-Privilege Rule: Allow the native debian-tor system profile out
sudo iptables -A OUTPUT -m owner --uid-owner debian-tor -j ACCEPT

# Interception Engine: Capture UDP Port 53 DNS and redirect to 5353
sudo iptables -t nat -A PREROUTING -i ens20 -p udp --dport 53 -j REDIRECT --to-ports 5353

# Interception Engine: Capture all standard TCP web traffic and force to 9040
sudo iptables -t nat -A PREROUTING -i ens20 -p tcp -j REDIRECT --to-ports 9040

# Absolute Leak-Guard Killswitch: Deny all raw packet transit
sudo iptables -P FORWARD DROP
```

Make the active rule configurations non-volatile and survival-proof over system power-cycling:

```
sudo apt install iptables-persistent -y
sudo netfilter-persistent save
```

4. Machine 3 (VM3) — Hardened Workstation Configuration

VM3 acts as the operational sandbox (user: `analysis`). It possesses zero raw routing permissions. It operates under the structural assumption that its default gate is the transparent proxy interface.

4.1 Strategic Interface Separation Netplan Definition

To eliminate asymmetric metric calculations or direct infrastructure leakage, the secondary setup wire (`ens18`) must stripped of default routing status entirely. It can only retain an internal LAN mapping sequence for manual patch tasks.

```
sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  version: 2
  ethernets:
    ens18:
      addresses:
        - 192.168.1.50/24  # Example maintenance allocation only
    ens19:
      addresses:
        - 10.0.2.2/24
      routes:
        - to: default
          via: 10.0.2.1
      nameservers:
        addresses:
          - 10.0.2.1
```

```
sudo netplan apply
```

4.2 Manual Resolution Path Override

Enforce the absolute mapping parameters to prevent automated DHCP sweeps from poisoning the local name resolution targets:

```
sudo nano /etc/resolv.conf
```

Clear out any configuration entries and append exactly one assertion statement:

```
nameserver 10.0.2.1
```

4.3 Outbound Pipeline Execution Clearances

Ensure that no secondary application profiles (such as ufw or local firewalls) are silently clamping down on outbound communication frames:

```
sudo iptables -F
sudo iptables -X
sudo iptables -t nat -F
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P INPUT ACCEPT
```

5. Global Infrastructure Diagnostics & Operational Debugging Matrix

When communication breaks down across the multi-node boundary, utilize this linear diagnostic cascade to determine exactly where the packet logic fails.

5.1 VM1 (VPN Gateway) Diagnostic Operations

Diagnostic Objective	Terminal Testing Command Syntax	Expected Validation Outcome
Verify Interface State	`ip link show wg0`	Must display state `UNKNOWN` or `UP` with correct MTU parameters.
Cryptographic Handshake Check	`sudo wg show`	Must show active peer mapping along with "latest handshake" timestamps.
Verify UFW NAT Tracking	`sudo ufw status verbose`	Must confirm forwarding rules are active with default deny parameters.
Trace Real-Time Packets	`sudo tcpdump -ni ens19 icmp`	Traces if VM2 ping packets are successfully striking the gateway.

5.2 VM2 (Tor Middlebox) Diagnostic Operations

Diagnostic Objective	Terminal Testing Command Syntax	Expected Validation Outcome
Upstream Gateway Verification	`ping -c 3 10.0.1.1`	Must register successful ICMP return loops from VM1.
External Tunnel Route Check	`ping -c 3 8.8.8.8`	Verifies VM1 is successfully performing Network Address Translation out to VPN.
Tor Bootstrap Status Monitoring	`sudo journalctl -u tor@default -n 50 --no-pager`	Must output the structural line: `Bootstrapped 100% (done): Done`.
Verify System Socket State	`sudo ss -tulpn | grep tor`	Must show ports `10.0.2.1:9040` and `10.0.2.1:5353` actively in `LISTEN` status.
Audit Redirection Engine Counters	`sudo iptables -t nat -L -v -n`	Packet/Byte metric columns next to port 53/9040 rules must tick up under load.

5.3 VM3 (Hardened Workstation) Diagnostic Operations

Diagnostic Objective	Terminal Testing Command Syntax	Expected Validation Outcome
Determine Target Interface Map	`ip route get 1.1.1.1`	Must display explicitly: `1.1.1.1 via 10.0.2.1 dev ens19`.
Verify Isolated Local Link	`ping -c 3 10.0.2.1`	Confirms bidirectional communication with the interception gateway.
Active DNS Translation Check	`nslookup check.torproject.org`	Must return a valid address translation parsed strictly via the Tor engine.
The Ultimate Operational Validation	`wget -O- https://check.torproject.org/api/ip`	Must produce a valid JSON block displaying explicitly: `"IsTor":true`.

> **CRITICAL TRAFFIC ANALYSIS TRICK**
>
> If `wget` output on VM3 appears to stall out indefinitely without throwing a name resolution failure code, run `sudo iptables -t nat -L -v -n` on **VM2** in a separate terminal loop. If the counters on port `9040` are growing, the network handoff is operating perfectly, and Tor is actively processing the stream. The latency is purely standard Tor circuit negotiation overhead.